HYPERLIQUIDFLOW

TECHNICAL WHITEPAPER

Real-time trading intelligence for Hyperliquid perpetuals. Rule-based. No ML inference. No paid data feeds.

DOCUMENT

Version 1.0

April 2026

Confidential

V1.0

00 OVERVIEW

Abstract

FOR THE TRADER

HyperliquidFLOW watches a curated group of the best-performing wallets on Hyperliquid perpetuals and tells you what they are doing with their money right now. Every 60 seconds it snapshots every tracked wallet's open positions, compares them to the last snapshot, and fires alerts when the change matches one of 13 defined patterns. The patterns range from multiple top wallets piling into the same trade at once, to a single elite wallet quietly adding to a position while the price is flat. Each alert comes with an expected-value score built from that wallet's historical trade record.

FOR THE ENGINEER

The platform runs two data paths: a Vercel serverless cron (60s cycle, rotating 100-wallet window) and a GitHub Actions daily scan (full cohort scoring from 30-day closing fills). Wallet scores use a 35/25/25/15-weighted composite of Sharpe proxy, PnL consistency, drawdown score, and regime fit. Signal recipes operate on consecutive SnapshotPair deltas; R10 and R13 are async with KV state. EV enrichment uses a walk-the-book L2 slippage model on a $10K reference notional. All intelligence derives from the Hyperliquid public API. Known approximations are in Section 7.

00 CONTENTS

01 Introduction

02 Cohort Architecture

2.1 Wallet Discovery

2.2 Activation and Pruning

2.3 Tier Classification

03 Scoring Model

3.1 Data Foundation

3.2 Factor Definitions

3.3 Composite Score

3.4 Regime Detection

04 Signal Architecture

4.1 Snapshot Pair Model

4.2 Recipe Catalog
4.3 Whale Validation (R8)
4.4 Async Recipes (R10, R13)

05 EV Enrichment

06 Data Infrastructure

6.1 Dual-Path Architecture
6.2 Supabase Table Reference
6.3 KV Key Reference
6.4 Cron Budget

07 Known Limitations and Approximations

08 Glossary

01 SECTION

Introduction

The Hyperliquid perpetuals exchange publishes complete, real-time position and fill data for every wallet via a free public API. This makes a specific class of analysis possible: observe what the best traders are actually holding, track how those positions change cycle to cycle, and surface patterns before they resolve.

"Smart money on Hyperliquid" is operationally defined here as wallets that passed a statistical activation gate (win rate ≥52%, ≥30 trades in 30 days), maintain a composite score above 0.35 across four risk-adjusted performance factors, and have demonstrably survived the pruning cycle. The cohort is not curated by hand. It is rebuilt daily from the Hyperliquid leaderboard and supplementary fills-based discovery, then filtered by the scoring model.

The platform does not predict prices. It detects what high-scoring wallets are doing with their positions right now, and whether that behavior matches patterns historically associated with favorable outcomes. All computation is rule-based. No machine learning models are involved at any stage of the pipeline.

02 SECTION

Cohort Architecture

> **PLAIN ENGLISH**
>
> Every day the platform scans the Hyperliquid leaderboard, filters for consistently profitable wallets, and keeps a live list worth watching. Wallets that fall below the quality bar get cut automatically after each 60-second cycle.

2.1 Wallet Discovery

Discovery runs daily at 00:00 UTC via GitHub Actions. The primary source is a GET request to `stats-data.hyperliquid.xyz/Mainnet/leaderboard`. All addresses passing the pre-filter are upserted to the `wallets` table.

PRE-FILTER CRITERION	THRESHOLD
Month ROI	≥ 3%
Month PnL	≥ $5,000
All-time ROI	≥ 0%

A supplementary path mines fills from the top-10 coins by 24h volume (from `metaAndAssetCtxs`). Addresses appearing in those fills are run through the same pre-filter and upserted as new candidates, providing coverage for high-activity wallets that may not rank prominently on the leaderboard at scan time.

> **FALLBACK STATUS**
>
> If the leaderboard GET fails, the fallback path returns hardcoded placeholder addresses. Discovery falls back entirely to the fills-based supplementary path, which provides partial but incomplete coverage.

2.2 Activation and Pruning

Not every discovered wallet enters the active cohort. Activation requires `win_rate ≥ 0.52` and `trade_count_30d ≥ 30`, evaluated by the daily scan after backtest computation. New wallets not yet scored receive only a regime_fit-based score (factors 1-3 = 0) and no EV enrichment.

Pruning runs after each cron cycle via `after()`. A wallet is deactivated when its `overall_score` falls below 0.35. The top 4 wallets by score are always retained regardless of threshold.

2.3 Tier Classification

Account equity from `cohort_snapshots.account_value` determines tier on every snapshot write.

TIER	ACCOUNT EQUITY
Elite	≥ $5,000,000
Major	$1,000,000 to $4,999,999
Large	$500,000 to $999,999
Mid	$100,000 to $499,999
Small	$50,000 to $99,999
Micro	$1,000 to $49,999
Dust	Under $1,000

03 SECTION

Scoring Model

PLAIN ENGLISH

Each wallet gets a score from 0 to 1. The score answers: "How consistently profitable is this trader, and are they on the right side of the current market?" Higher is better. Scores below 0.35 get cut.

3.1 Data Foundation

All three history-based scoring factors read from `user_pnl_backtest.daily_pnls`: a 30-element array of per-day realized PnL built from closing fills only (where `closedPnl !== "0"`). Opening fills are excluded because they carry no realized PnL. Index 0 = 30 days ago, index 29 = today. Days with no closing activity remain 0.

3.2 Factor Definitions

FACTOR 1 — SHARPE PROXY 35%

```
raw_sharpe   = mean(daily_pnls) / (stddev(daily_pnls) + 0.0001)
sharpe_proxy = clamp(raw_sharpe / 3.0, 0, 1)
```

Epsilon (0.0001) prevents division by zero for flat wallets. The divisor 3.0 is the ceiling: raw Sharpe of 3 produces a score of 1.0. Negative Sharpes floor at 0.

FACTOR 2 — PNL CONSISTENCY 25%

```
Split daily_pnls into non-overlapping 7-day windows (including partial final window).
pnl_consistency = count(windows where sum > 0) / count(windows with at least one non-
zero day)
```

A 30-day array produces up to 5 windows. Windows where all values are zero are excluded from the denominator so sparse traders are not penalized for inactivity.

FACTOR 3 — DRAWDOWN SCORE `25%`

```
Build cumulative PnL series from daily_pnls.
For each peak > 0:
  drawdown_fraction = (peak - current_value) / (|peak| + 0.0001)
drawdown_score = clamp(1 - max(drawdown_fraction), 0, 1)
```

Peaks at or below zero are skipped. Wallets that never accumulated positive cumulative PnL have no meaningful drawdown to measure.

FACTOR 4 — REGIME FIT `15%`

```
bias = sum(signed_notional) / (total_notional + 0.0001)   // [-1, 1]

BULL:    regime_fit = clamp((bias + 1) / 2, 0, 1)   // rewards long bias
BEAR:    regime_fit = clamp((1 - bias) / 2, 0, 1)   // rewards short bias
RANGING: regime_fit = clamp(1 - |bias|, 0, 1)       // rewards neutral bias
```

Computed live each cycle from `assetPositions`. Wallets with no open positions return 0.5 (neutral).

3.3 Composite Score

35%	25%	25%	15%
SHARPE PROXY	PNL CONSISTENCY	DRAWDOWN SCORE	REGIME FIT

```
overall_score = 0.35 * sharpe_proxy + 0.25 * pnl_consistency + 0.25 * drawdown_score + 0.15 *
regime_fit
```

NOTE ON WEIGHTS

The 35/25/25/15 weighting is a calibration assumption, not an empirically optimized vector. A validation script (`scripts/validate-scoring-weights.ts`) checks correlation against 30-day EV scores but does not produce an optimized vector.

3.4 Regime Detection

Each cron cycle writes the current BTC mid to `btc_mid:{HH}` (UTC hour, zero-padded, TTL 25h). The 24h-prior price is read from the key for the same hour 24 hours earlier, giving a stable window that does not drift with cron timing variance. On KV miss, the fallback assumes current BTC mid was 1% lower 24 hours ago.

BTC 24H RETURN	REGIME	SCORING EFFECT
> +1.0%	BULL	Rewards long-biased wallets
< -1.0%	BEAR	Rewards short-biased wallets
Between -1% and +1%	RANGING	Rewards neutral wallets

Regime is BTC-only. No per-coin or altcoin regime tracking exists.

04 SECTION

Signal Architecture

> **PLAIN ENGLISH**
>
> Every 60 seconds the platform compares each wallet's current open positions to the last snapshot. If the change matches one of 13 defined patterns, it fires a signal. Think of it as a smoke detector, not a crystal ball.

4.1 Snapshot Pair Model

Each cron cycle produces a `SnapshotPair` per wallet: `curr` (this cycle's `clearinghouseState`) and `prev` (prior cycle's snapshot from Supabase, or null on first run). Signal recipes diff `curr.positions` against `prev.positions` and skip null pairs. Snapshot data is open positions and margin metrics only. Realized PnL comes exclusively from `userFills.closedPnl`.

4.2 Recipe Catalog

ID	DISPLAY NAME	TYPE	LEVEL	TRIGGER CONDITION	NOTES
`momentum_stack`	Whale Convergence	ENTRY	Cohort	3+ wallets add combined $500K+ same direction within 5 min	
`divergence_squeeze`	Silent Loading	ALERT	Per-wallet	Exposure rising, price flat (<0.5%), liq buffer <15%	
`accumulation_reentry`	Dip Conviction	SCALE IN	Per-wallet	Score ≥0.65 wallet adds after coin drops >8% from 4h high	
`rotation_carry`	Funded Edge	ENTRY	Per-wallet	New position with funding >0.03%/hr; requires ≥10 prior signals for win-rate gate	*Cold-start suppressed*
`liq_rebound`	Liquidation Flush	ALERT	Cohort	Cohort notional drops >5% on a coin AND price moves >2% vs prior cycle	*Approximated; see §7*
`streak_continuation`	Hot Streak	ALERT	Per-wallet	Score ≥0.65 wallet on 5+ win streak with normalized Sharpe proxy ≥0.6	*Daily-scan frequency*
`funding_divergence`	Smart Money vs. Retail	ALERT	Cohort	Cohort direction opposite to funding rate sign; funding >0.05%/hr	
`whale_validated`	Alpha Confirmation	ENTRY	Meta	R1-R7 or R10-R13 signal corroborated by 3+ wallets with score ≥0.75, fresh entry or >5% size growth	*Dedup: original removed*
`anti_whale_trap`	Smart Exit Signal	EXIT	Per-wallet	Score ≥0.65 wallet cuts total exposure ≥20% with regime_fit <0.35	
`position_aging`	Patience Trap	ALERT	Per-wallet	Score ≥0.65 wallet underwater >5% of position for 2+ consecutive cycles, not reducing	*Async; KV state*
`concentration_risk`	Crowded Coin	ALERT	Cohort	One coin >60% of total cohort notional	*Direction: null*
`wallet_churn`	Coordinated Exit	EXIT	Cohort	3+ wallets reducing same coin by combined $500K+ within 5 min	
`funding_trend`	Funding Surge	ALERT	Cohort	Coin funding rising 3+ consecutive cycles and above 0.03%/hr	*Async; KV state; SHORT*

Cohort-level recipes (`liq_rebound`, `concentration_risk`, `funding_divergence`, `funding_trend`) have no single attributable wallet. They are persisted to `signals_history` with `cohort_level: true` in metadata. EV score is always null for cohort-level events.

MOMENTUM DETECTION

- **R1, R3, R12** detect directional commitment within a narrow window
- R1 fires on coordinated addition, R12 on coordinated reduction
- R3 narrows to high-score wallets buying dips

STRUCTURAL RISK

- **R2** identifies wallets accumulating under liquidity pressure
- **R11** identifies macro concentration that could amplify drawdown

CARRY AND FUNDING

- **R4** enters when carry is positive and historical win-rate supports it
- **R7** uses funding sign as a retail crowd proxy
- **R13** detects trend acceleration in the funding rate across cycles

QUALITY AND LIFECYCLE

- **R6** watches high-quality wallets on hot streaks
- **R8** validates signals from all other recipes
- **R9** watches high-score wallets defensively exit
- **R10** tracks positions underwater for multiple cycles
- **R5** approximates forced liquidation cascades

4.3 Whale Validation (R8)

Recipe 8 re-emits signals from any other recipe when the coin and direction have fresh activity from 3 or more wallets with `overall_score ≥ 0.75`. "Fresh" means: position opened this cycle, or position size grew by more than 5% since the previous cycle. Whales holding unchanged week-old positions do not qualify. When a signal is re-emitted as `whale_validated`, the original is removed from the deduplicated set.

4.4 Async Recipes (R10, R13)

R10 reads and writes `cohort:underwater_counts` (a JSON map of `walletId:coin` to consecutive-cycle underwater count, TTL 25h). It fires when the count reaches 2. The count resets when the position improves, the wallet reduces size by more than 5%, or the position closes.

R13 reads and writes `market:funding_history:{coin}` (a 4-element rolling array of funding rate readings per coin, TTL 25h). It fires when current funding is above 0.03%/hr and the last three readings are strictly increasing. Both run in parallel with synchronous recipes inside `runSignalLab()`.

05 SECTION

EV Enrichment

> **PLAIN ENGLISH**
>
> Each signal gets a score from 0 to 1 based on that wallet's historical track record. A score above 0.6 is considered worth acting on. Cohort-level signals get no score because they do not belong to a single wallet.

5.1 Formula

```
EV       = (win_probability * avg_win_usd) - (loss_probability * avg_loss_usd) -
trade_cost_usd
ev_score = clamp(EV / (5 * avg_loss_usd), 0, 1)

A 5x edge (EV = 5 * avg_loss) maps to score 1.0. Scores above 0.6 are actionable.
```

`win_probability`, `avg_win_usd`, and `avg_loss_usd` come from the wallet's `user_pnl_backtest` row. `trade_cost_usd` = taker fee (0.035% of notional) + slippage estimate. Reference notional is $10,000 for all signals regardless of wallet tier or actual position size.

5.2 Slippage Model

```
Walk the appropriate book side (buys consume asks, sells consume bids).
For each level: consume min(remaining_notional, level_notional).
slippage = |avg_fill_price / mid_price - 1| * notional

mid_price = (best_bid + best_ask) / 2 at fetch time.
If book exhausted before fill complete, extrapolate from the last level price.
```

L2 books are fetched for the top 10 coins by cohort notional per cycle. Signals on other coins use fee-only cost (slippage = 0).

5.3 Null Conditions

EV score is null for: cohort-level signals (`liq_rebound`, `funding_divergence`, `concentration_risk`, `funding_trend`); wallets without a `user_pnl_backtest` row; wallets with a perfect win rate (no loss history); and any wallet where `avg_win_usd = 0`, `avg_loss_usd = 0`, `win_probability = 0`, or `win_probability = 1`.

06 SECTION

Data Infrastructure

> **PLAIN ENGLISH**
>
> The platform runs two jobs: a fast 60-second cron that watches wallets and fires signals, and a slow daily scan that rescores everyone. Data lives in Supabase (database) and Vercel KV (fast cache). The cron must finish in under 10 seconds.

6.1 Dual-Path Architecture

PATH A — VERCEL CRON (EVERY 60 SECONDS)

1. Load active wallets. Read KV offset (`cohort:active_offset`), select rotating 100-wallet window, increment and store offset.
2. `fetchBatchClearinghouseStates()` for the window: up to 10 concurrent requests, API weight 2 each.
3. Parallel fetch: `fetchAllMids()`, `fetchMetaAndAssetCtxs()`, `market:prior_mids` from KV.
4. Regime detection: write current BTC mid to `btc_mid:{H}`, read `btc_mid:{H-24}` for 24h prior.
5. Load prior `cohort_snapshots` from Supabase (one per wallet in the window).
6. Load `user_positions` rows for scoring factors 1-3.
7. Score each wallet. Fire-and-forget `saveOneWalletSnapshot()` per wallet.
8. Fetch L2 books and 5m candles (4h window) for top 10 coins by cohort notional.
9. Load last 50 rows of `recipe_performance` for Recipe 4 win-rate gate.
10. Run `runSignalCalc()`: all 13 recipes, EV enrichment, intraday perf cache, persist to `signals_history`.
11. Write `CohortCachePayload` to `cohort:latest` (TTL 120s). Write `market:prior_mids` (TTL 24h, fire-and-forget).

12 Background via `after()`: `pruneUnderperformers()`

PATH B — GITHUB ACTIONS (DAILY, 00:00 UTC)

1. GET leaderboard; apply pre-filter (month ROI ≥3%, month PnL ≥$5K, all-time ROI ≥0).
2. Upsert all qualifying addresses to `wallets` table.
3. Supplementary discovery: mine fills from top-10 coins by 24h volume; apply same pre-filter.
4. Score batch: active wallets + inactive candidates (stalest first, up to 5,000 total), concurrency 3, 600ms delay per slot. Per wallet: `userFillsByTime` (last 30 days), compute backtest, upsert `user_pnl_backtest`
5. Aggregate `signals_history` (last 30 days) by recipe, insert into `recipe_performance`
6. Write `scan-summary.json` artifact

6.2 Supabase Table Reference

TABLE	WRITTEN BY	READ BY
`wallets`	Daily scan	Cron (load active), pruning
`user_pnl_backtest`	Daily scan	Cron (scoring factors 1-3), EV enrichment
`cohort_snapshots`	Cron (per cycle)	Signal recipes (prev snapshot), scoring
`signals_history`	Cron via runSignalLab()	KV payload, Edge page, recipe_performance, intraday perf
`recipe_performance`	Daily scan	Recipe 4 win-rate gate, Edge page

6.3 KV Key Reference

KEY	TTL	PURPOSE
`cohort:active`	120s	Primary cohort cache; read by client poll
`cohort:active:fallback`	24h	Fallback if primary expires between cron cycles
`cohort:cycle_offset`	25h	Rotating window pointer for 100-wallet batching
`btc_mid:{HH}`	25h	Per-hour BTC mid; 24 keys (00-23); stable 24h regime window
`market:prior_mids`	10 min	allMids from prior cycle; R5 price confirmation input
`market:funding_history:{coin}`	25h	Last 4 funding readings per coin; R13 trend state
`cohort:underwater_counts`	25h	Per wallet+coin consecutive underwater count; R10 state
`recipe:intraday_perf`	7h	6h rolling avg EV score per recipe; intraday supplement

6.4 Cron Budget

The Vercel free tier enforces a ~10-second execution ceiling on cron functions. Heavy scoring and full-cohort backtesting run in GitHub Actions (Path B). Snapshot writes are fire-and-forget. Pruning runs via `after()` after the response is returned. The rotating 100-wallet window bounds API call count per cycle as the cohort grows.

07 SECTION

Known Limitations and Approximations

> **PLAIN ENGLISH**
>
> This section is the honest part. The system works, but some things it says are estimates or have gaps. Here is the complete list with no softening.

01 Recipe 5 (liq_rebound) is approximated

The recipe fires when cohort notional on a coin drops more than 5% and price moves more than 2% versus the prior cycle mid. This cannot confirm the reduction was involuntary. True detection requires the Hyperliquid WebSocket liquidation stream, which is incompatible with Vercel serverless (tagged Phase 3). The recipe silently skips its first cycle after deploy until `market:prior_mids` is populated.

02 Recipe 6 (streak_continuation) win streaks update daily, not intraday

Streak data in `user_pnl_backtest` is written by the daily scan at 00:00 UTC. A streak broken during trading hours is not reflected until the next scan. The UI shows a staleness indicator; the underlying fix remains open.

03 Recipe 4 (rotation_carry) is suppressed on fresh deploys

Requires at least 10 prior `rotation_carry` signals in `recipe_performance` before the 60% win-rate gate activates. The recipe is silent on new deployments until that history accumulates.

04 EV scores are not scaled to actual position size

The reference notional is $10,000 regardless of wallet tier or actual position size. Wallet-tier-adjusted notional would produce more accurate cost estimates.

05 New wallets have null EV scores until the daily scan processes them

Wallets discovered via leaderboard or fills mining but not yet scored have no `user_pnl_backtest` row. Their signals fire with `ev_score: null`.

06 L2 slippage estimated for the top 10 coins only

Signals on coins outside the top 10 by cohort notional use fee-only cost (slippage = 0). This understates cost for less liquid coins.

07 Regime is BTC-only

A wallet correctly positioned in a trending altcoin during a BTC-ranging environment is penalized on `regime_fit`. No per-coin or sector-level regime tracking exists.

08 **The leaderboard discovery fallback is non-functional**

If the leaderboard GET fails, the system falls back to hardcoded placeholder addresses. Discovery relies entirely on the fills-based supplementary path, which provides partial but incomplete coverage.

09 **Scoring weights are assumed, not empirically optimized**

The 35/25/25/15 weight vector is a starting point based on risk-adjusted performance intuition. The validation script checks correlation but does not produce an optimized vector.

08 SECTION

Glossary

SnapshotPair

A pair of consecutive `cohort_snapshots` rows for a single wallet: `curr` (this cycle) and `prev` (prior cycle, or null on first run). Signal recipes operate on SnapshotPairs and emit zero or more `SignalEvent` objects.

---

closing fill

A trade fill where `closedPnl ≠ "0"`. Opening fills always carry `closedPnl = "0"` and are excluded from all backtest computation.

---

daily_pnls array

A 30-element array of realized PnL per UTC day. Index 0 = 30 days ago, index 29 = today. Built from closing fills. Stored in `user_pnl_backtest`. Source for Sharpe proxy, PnL consistency, and drawdown score.

---

sharpe_proxy (normalized)

`clamp(mean(daily_pnls) / (stddev(daily_pnls) + 0.0001) / 3.0, 0, 1)`. A raw Sharpe of 3 maps to 1.0. The normalized form is used in scoring; the raw annualized form is available via `annualisedSharpe()` for display.

---

liq_buffer_pct

`(account_value - total_margin_used) / account_value`. Fraction of account value not committed to margin. Lower values indicate proximity to liquidation. Stored per snapshot in `cohort_snapshots`.

---

cohort-level event

A signal emitted by `liq_rebound`, `funding_divergence`, `concentration_risk`, or `funding_trend`. These evaluate cohort aggregate state, not individual wallet behavior. Persisted with `cohort_level: true` in metadata. EV score is always null.

EV score

`clamp(EV / (5 * avg_loss_usd), 0, 1)`, where
`EV = (win_prob * avg_win) - (loss_prob * avg_loss) - trade_cost`. A 5x expected edge maps to 1.0. Scores above 0.6 are considered actionable.

---

regime_fit

The fourth scoring factor. Measures alignment between a wallet's current directional bias (net signed notional / total notional) and the detected BTC regime (BULL/BEAR/RANGING). Flat wallets return 0.5.